

UBER DEMAND–SUPPLY GAP ANALYSIS

Project Documentation Report

Submitted by

Aishwarya Bharathy Babu

6,745

Total Ride Records

41.97%

Completion Rate

58.03%

Failure Rate

5 Days

Date Range

● Microsoft Excel

● SQL — DuckDB

● Python EDA

Dataset: Uber Ride Requests | Jul 11–15, 2016 | 6,745 Records

Business
Objective

Data Collection

Cleaning &
Preprocessing

Python EDA

SQL Analysis

Excel Dashboard

Insights & Recs

1 · Business Objective

This project conducts a comprehensive analysis of Uber ride request data to identify root causes of the demand–supply gap and propose data-driven interventions to improve service efficiency and customer satisfaction.

- Analyse 6,745 ride requests across a 5-day window (Jul 11–15, 2016)
- Identify the demand–supply gap — where rider demand exceeds driver supply
- Distinguish supply-side failures (No Cars Available) from cancellations
- Provide actionable recommendations to optimise driver allocation and completion rates

2 · Business Problem

Uber faces persistent operational challenges causing revenue loss and poor customer experience:

Problem	Business Impact
Only 41.97% of rides complete	58% failure rate — majority of requests go unserved
39.3% No Cars Available	2,650 rides get no driver — direct revenue loss
18.74% Cancellation rate	1,264 rides cancelled — customer trust erosion
Airport 52.9% no-car rate	Over half of all Airport pickups fail — critical supply gap
Completion rate declining	44.0% → 40.6% in 5 days; trending below 35% in two weeks

3 · Dataset Understanding

Raw dataset: **Uber_Request_Data.xlsx** — 6,745 records, 6 original columns. Cleaned dataset: **Uber_Request.xlsx** — same records plus 4 engineered features.

Raw Dataset — 6 Original Columns		Engineered Columns (Cleaned)	
Request id	Unique ride identifier	Request hour	Extracted from timestamp (.dt.hour)
Pickup point	Airport or City	Request date	Date for daily trend analysis
Driver id	NULL if no driver assigned	Trip duration (mins)	Drop – Request time in minutes
Status	Trip Completed / Cancelled / No Cars Available	Has driver	Boolean: True if Driver id not NULL
Request timestamp	Date & time of request		
Drop timestamp	NULL if ride failed		

4-5 · Missing Values — Analysis & Actual Reason

Column	Missing	%	Business Reason
Driver id	2,650	39.3%	No Cars Available → no driver dispatched
Drop timestamp	3,914	58.0%	Cancelled or No Cars → no trip completion
Trip duration (mins)	3,914	58.0%	Cannot compute without a Drop timestamp

These are business-valid missing values — NOT data errors.

→ No Cars Available: no driver assigned → Driver id is legitimately NULL

→ Cancelled / No Cars: ride never completed → Drop timestamp is legitimately NULL

Removing these rows would delete all failed-ride data — the core subject of this analysis.

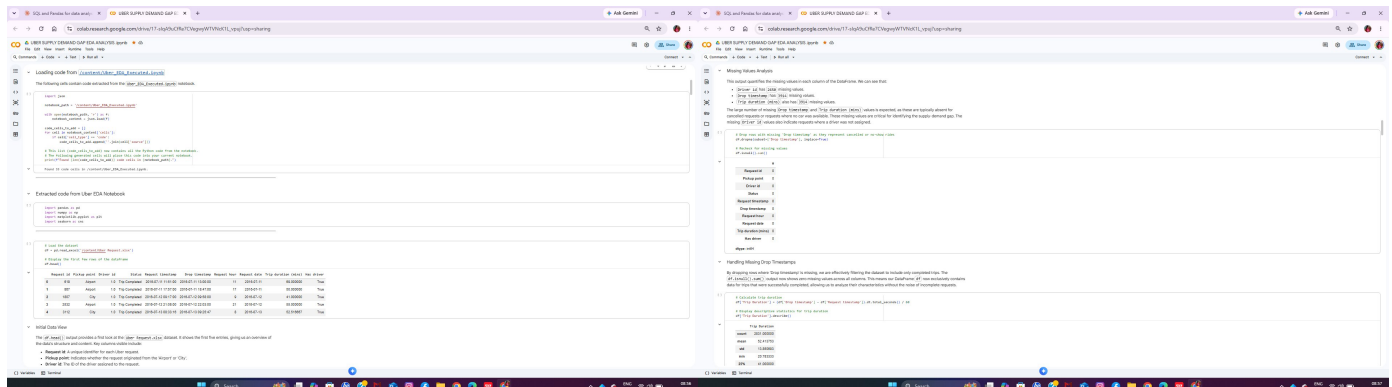
6 · How Missing Values Were Handled

Column	Treatment	Justification
Driver id	Retained as NULL	Valid state — ride had no driver
Drop timestamp	Retained as NULL	Represents the failed ride — needed for analysis
Trip duration (mins)	NULL for fails; calculated for completes	Duration valid only for 2,831 completed rides

7 · Data Cleaning & Preprocessing

#	Task	Method / Detail
1	Timestamp Conversion	pd.to_datetime() with errors='coerce' → datetime64[ns]
2	Hour Extraction	Request hour = .dt.hour (integer 0–23)
3	Date Extraction	Request date = .dt.date (for 5-day trend analysis)
4	Duration Calculation	Trip duration = (Drop – Request).dt.total_seconds() / 60
5	Boolean Driver Flag	Has driver = True if Driver id not NULL, else False

Dataset Preview · Raw Data vs Cleaned Data



The screenshot displays a Jupyter Notebook in Google Colab with the following sections:

- Loading data from 'uber_2016_07_11_12.csv':** The code uses `pd.read_csv()` to load the data. The output shows the first few rows of the raw data.
- Extracted data from Uber EDA iterations:** The code uses `pd.to_datetime()` to convert the 'Request' and 'Drop' columns to datetime format. The output shows the first few rows of the cleaned data.
- Initial Data Size:** The code uses `len(df)` to check the number of rows. The output shows the initial data size.
- Handling Missing Drop Timestamps:** The code uses `df['Drop'] = df['Drop'].fillna(df['Request'])` to handle missing drop timestamps. The output shows the first few rows of the cleaned data.

The final output shows a preview of the cleaned data with columns: Request, Drop, Request hour, Request date, Drop hour, Drop date, Trip duration, and Has driver.

Figure 1: Raw dataset — `df.head()` output, original 6 columns

Figure 2: Cleaned dataset — `df.info()` output, 10 columns after preprocessing

Before vs After Preprocessing

Attribute	Raw Dataset	Cleaned Dataset
Columns	6	10 (4 engineered columns added)
Timestamps	String / mixed format	Standardised datetime64[ns]
Request Hour	Not present	Extracted (integer 0–23)
Trip Duration	Not present	Calculated in minutes (float64)
Has Driver	Not present	Boolean flag (True / False)
Total Records	6,745	6,745 — no rows removed

8 · Python EDA — Exploratory Data Analysis

EDA was performed in Google Colab using **pandas**, **matplotlib**, and **seaborn** on the cleaned dataset, covering univariate distributions, bivariate cross-tabs, temporal patterns, and correlation mapping.

Task	Description
Data Loading & Inspection	<code>df.head()</code> , <code>df.info()</code> , <code>df.describe()</code> — structure and statistics
Missing Values Audit	<code>df.isnull().sum()</code> — quantified and validated business logic
Feature Engineering	Extracted hour, date, day-of-week; computed trip duration
Univariate Analysis	Frequency distributions for Status, Pickup Point, Request Hour
Distribution & Box Plots	Histograms (50 bins + KDE) and box plots by hour, status, pickup
Bivariate Cross-Tab	Status breakdown by pickup point and by hour of day
Time Series Trend	Daily completed request volume across the 5-day window
Correlation Heatmap	Seaborn coolwarm matrix of all numerical features

Chart 1 — Distribution of Trip Duration

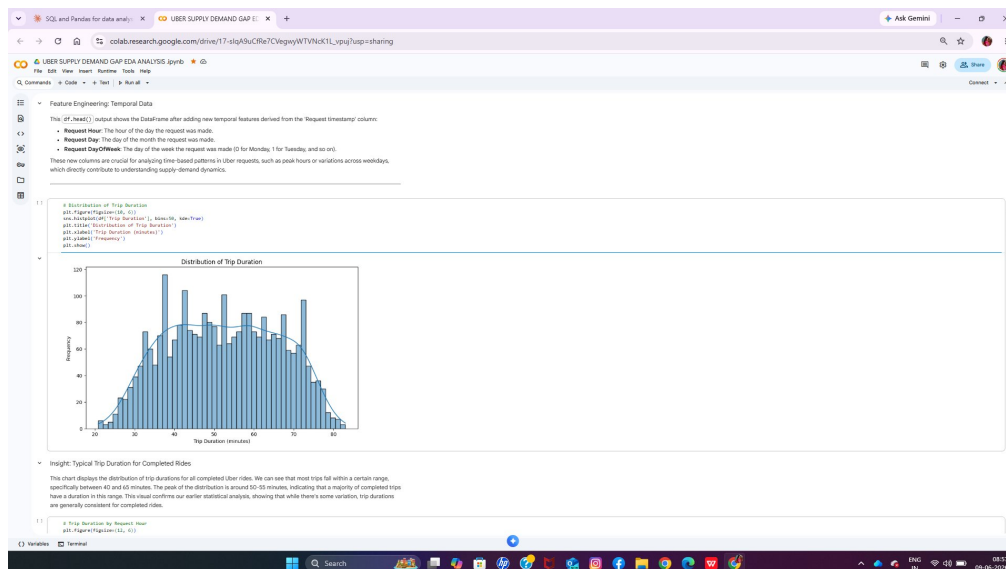


Figure 3: Histogram + KDE — Trip Duration for 2,831 completed rides

Most trips cluster between 40–65 min; peak at 50–55 min. Mean: 52.4 min | Std Dev: 13.85 min
27% CoV → trip times are unpredictable (21–83 min range). Right-skewed; median is more reliable.

8 cont. · Python EDA — Chart Insights Continued

Chart 2 — Number of Completed Requests by Hour of Day

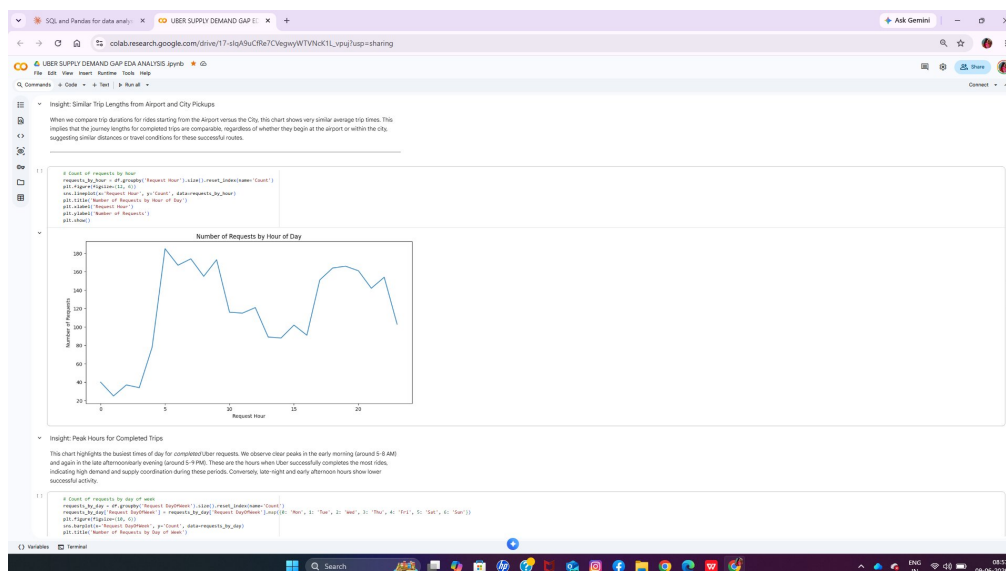


Figure 4: Line chart — Completed requests per hour showing twin demand peaks

Two demand peaks: early morning (5–8h) and evening (17–20h) for completed rides.
 Evening peak has highest demand but worst no-car failures (confirmed by SQL) — supply collapses at peak.
 Mid-day (10–14h) has the lowest failure rates — the operational benchmark window.

Chart 3 — Trip Duration by Pickup Point

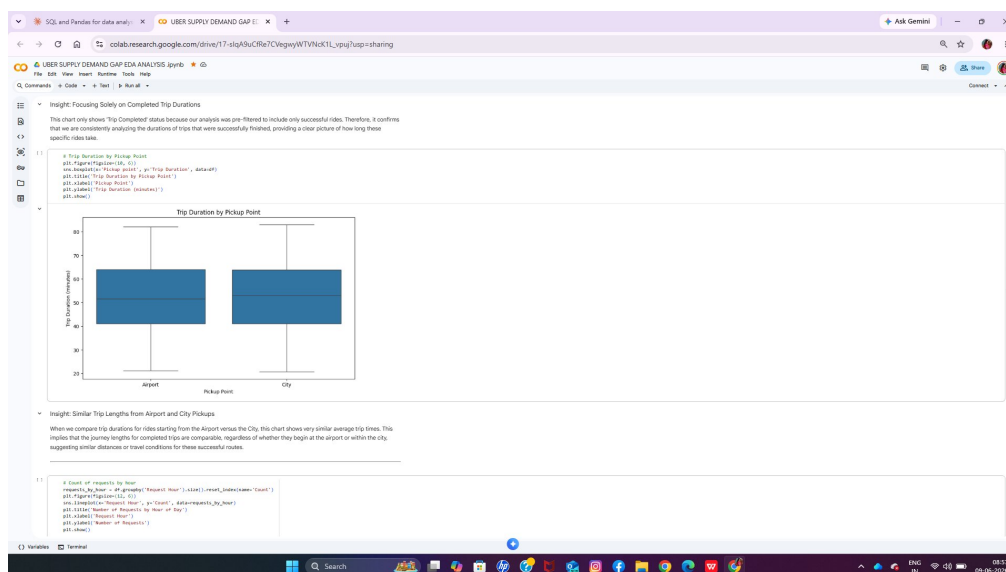


Figure 5: Box plot — Trip Duration for Airport vs City pickups

Airport avg: 52.24 min | City avg: 52.57 min | Both IQR: 41–64 min | Both Std Dev: ~13.85
 Virtually identical profiles → shared corridor (Airport ↔ City), not independent route networks.
 Once a driver is assigned, service quality is equal regardless of pickup location.

8 cont. · Python EDA — Correlation Heatmap

Chart 4 — Correlation Matrix of Numerical Features

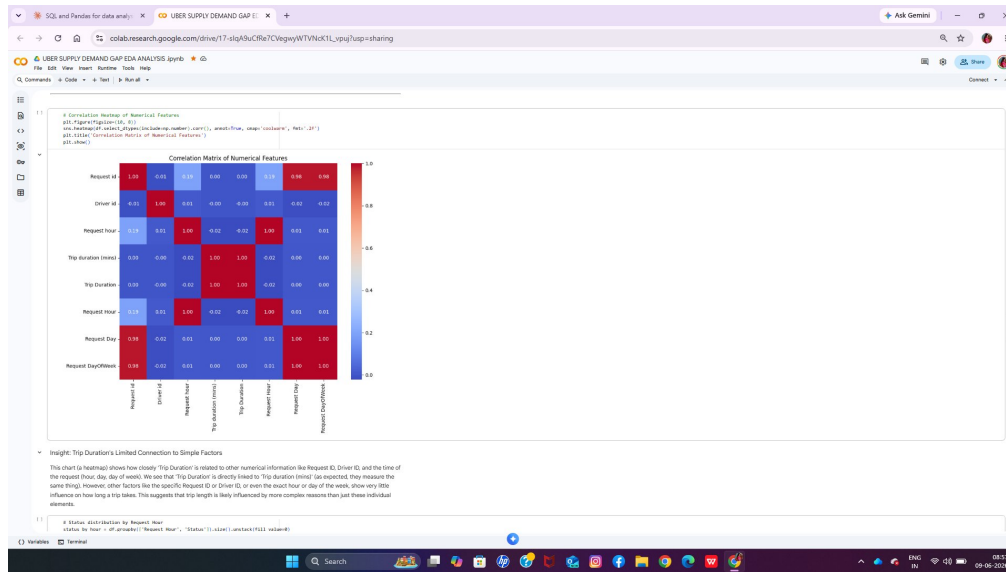


Figure 6: Seaborn coolwarm heatmap — correlation matrix of numerical features

Trip Duration has near-zero correlation with Request Hour (0.00), Driver ID (−0.02), Request Day (0.00).
Duration is driven by route distance alone — not time of day, driver identity, or day of week.
Request Day & DayOfWeek correlate perfectly (1.00) — both derived from the same timestamp.

9 · SQL Analysis — Query Overview

8 structured **DuckDB** queries were executed to generate business-focused insights across demand patterns, supply gaps, failure zones, and temporal trends.

Query	Analysis Focus
Q1	Overall Trip Status Distribution — 3-way outcome baseline across 6,745 rides
Q2	Demand & Outcome by Pickup Location — Airport vs City failure mode comparison
Q3	24-Hour Demand Cycle — peak demand, cancellation spikes, no-car surges
Q4	Time-Slot × Pickup Matrix — when and where service quality diverges most
Q5	Top 12 Supply Gap Slots — worst hour–location combos by no-driver rate
Q6	5-Day Daily Volume & Outcome Trend — demand escalation vs supply stagnation
Q7	Trip Duration Statistics — avg, min, max, std dev, percentiles by pickup
Q8	Critical Failure Zones (>65%) — CTE triage of highest-failure time×location slots

9 cont. · SQL Analysis — Key Insights

Insight 1 — Overall Trip Status: Only 41.97% Complete (Q1)

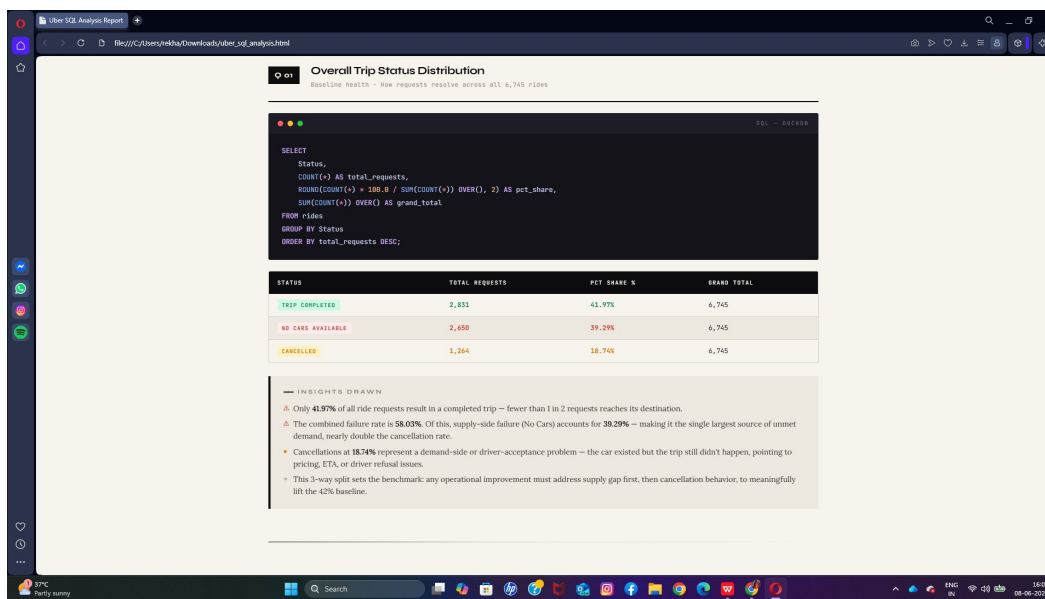


Figure 7: Q1 — Overall Trip Status Distribution across 6,745 rides

Trip Completed: 2,831 (41.97%) | No Cars Available: 2,650 (39.29%) | Cancelled: 1,264 (18.74%)
 Only 1 in 2 ride requests reaches its destination — systemic failure, not an occasional glitch.
 No Cars Available is the #1 failure mode, nearly double the cancellation rate.

Insight 2 — Airport vs City: Diverging Failure Modes (Q2)

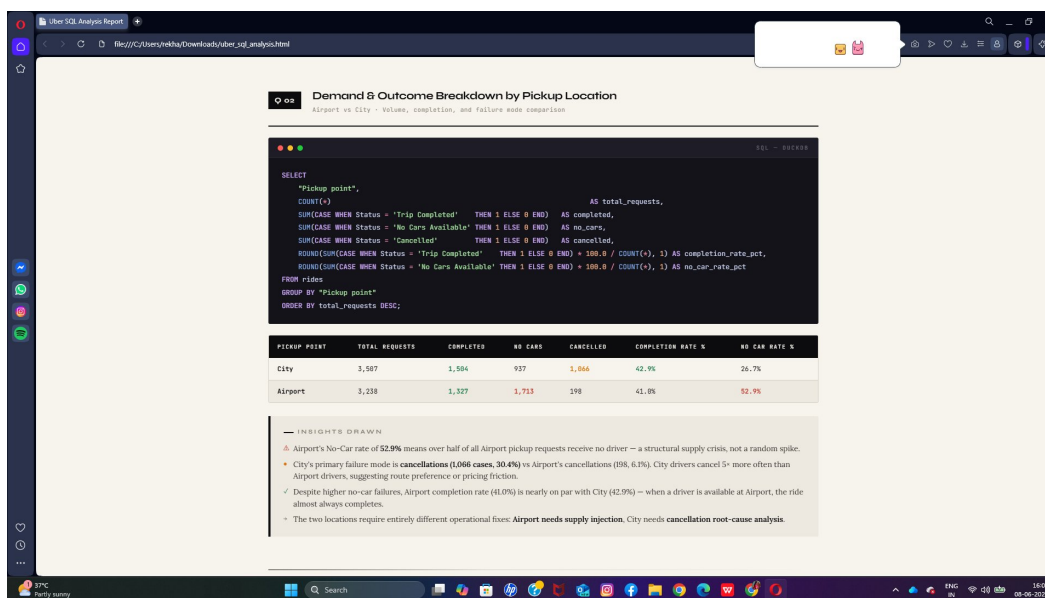


Figure 8: Q2 — Demand & Outcome Breakdown by Pickup Location

Airport: 3,238 requests | 52.9% No-Car rate | 41.0% completion
 City: 3,507 requests | 26.7% No-Car rate | 1,066 cancellations (5× Airport's 198)
 Different root causes → different fixes: Airport = supply injection; City = cancellation enforcement.

9 cont. · SQL Analysis — Supply Gap & Critical Failures

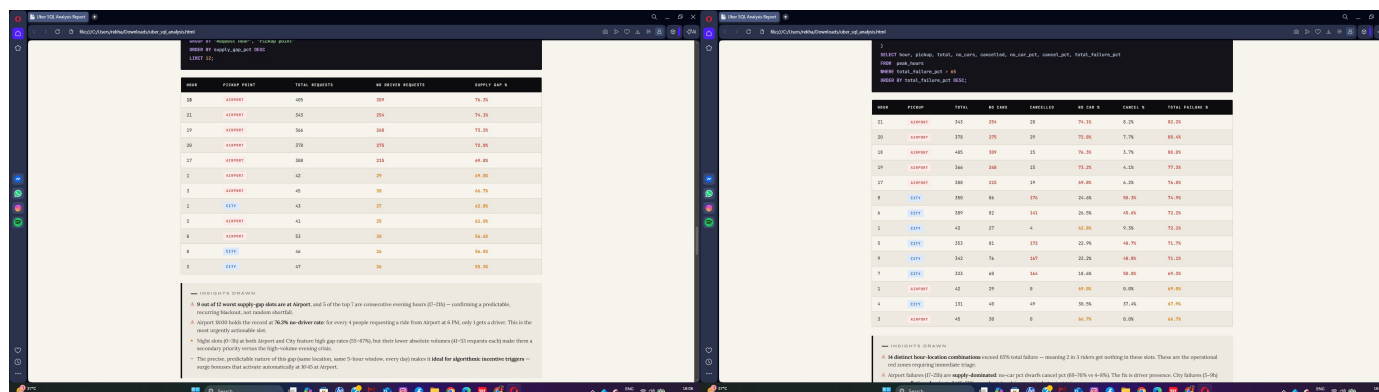


Figure 9: Q5 — Top 12 Supply Gap Slots by no-driver rate

Figure 10: Q8 — Critical Failure Zones (>65% total failure rate)

Supply Gap (Q5): Airport Hour 18 — 76.3% no-driver rate. 9 of top 12 worst slots are Airport. 5 consecutive evening hours (17–21h) confirm a recurring, predictable daily supply blackout.

Critical Failures (Q8): 14 hour-location combos exceed 65% total failure.

Airport 21h: 82.2% failure — only 1 in 5 late-night Airport requests completes.

Airport failures = supply-dominated (69–76% no-car). City failures = cancellation-dominated (45–51%).

10 · Excel Dashboard

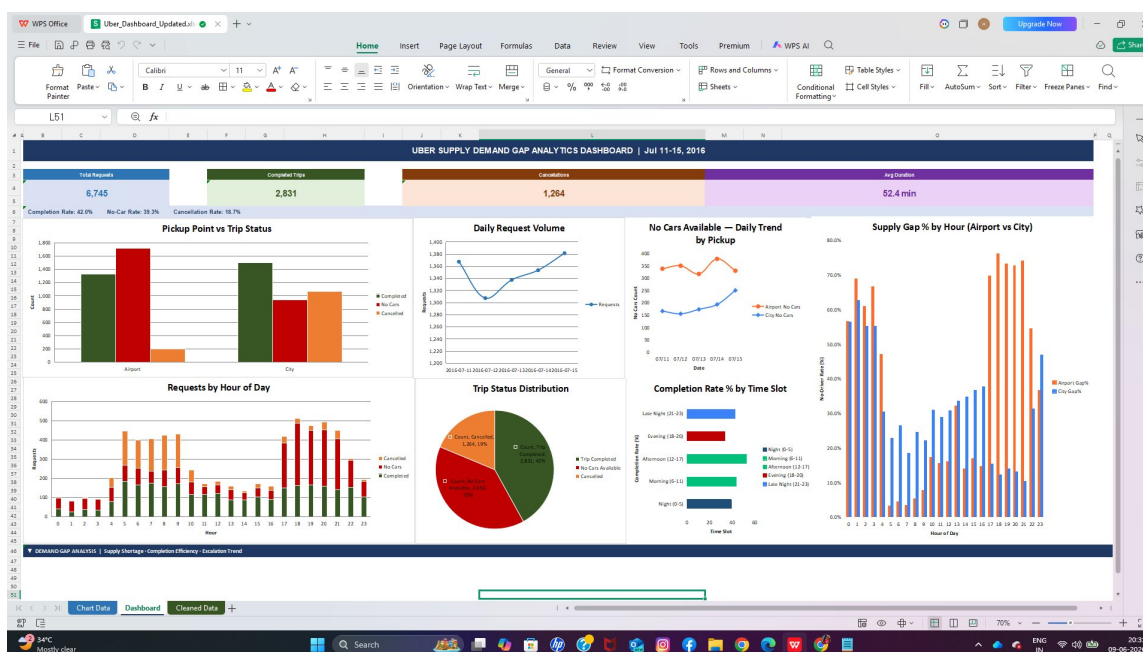


Figure 11: Uber Supply Demand Gap Analytics Dashboard — Jul 11–15, 2016

6,745 Total Requests	2,831 Completed	42.0% Completion Rate	39.3% No-Car Rate	18.7% Cancel Rate	52.4m Avg Duration
--------------------------------	---------------------------	---------------------------------	-----------------------------	-----------------------------	------------------------------

11 · Project Deliverables

Deliverable	Tool	Description
Raw Dataset	Excel	Uber_Request_Data.xlsx — original 6-column data
Cleaned Dataset	Excel/Python	Uber_Request.xlsx — 10 columns with engineered features
Python EDA Notebook	Google Colab	Cleaning, feature engineering, 8+ visualisations
SQL Query Analysis	DuckDB	8 business-focused queries with results & insights
Excel Dashboard	MS Excel	Interactive pivot dashboard with 7 charts + KPI cards
Project Documentation	PDF	This report — full project flow documentation
Presentation	PowerPoint	10-minute deck with screenshots and recommendations

12 · Business Recommendations

Recommendation	Target	Key Action	Expected Outcome
Airport Evening Supply Surge	Airport 17–21h	Surge incentives from 16:45; guarantee 30 drivers at Airport peak	No-car rate: 73% → <35%
City Morning Cancellation Fix	City 5–9h	Penalty for >15% cancel rate; short-trip incentive bonuses	Cancel rate: 45% → <20%
Dynamic Driver Allocation Model	All slots	ML hourly forecasting; auto pre-positioning 30 min ahead	Completion: 42% → 60%+

Conclusion · Summary of Key Findings

Root Cause	Airport evening supply shortage (17–21h, 55–76% no-car) is the single biggest lever — fixing this could add ~700 completed trips daily.
Dual Patterns	Airport failures are supply-driven; City failures are cancellation-driven. One-size-fits-all fixes will not work.
Escalating	Completion rate: 44.0% → 40.6% in 5 days. Without intervention, rates fall below 35% within two weeks.
Gold Standard	Airport Morning (6–11h) achieves 84.4% completion — proof the operation CAN perform. Replicate those conditions.
Data-Ready	Patterns are predictable, recurring, time/location-specific — ideal for algorithmic surge pricing and automated dispatch.